

Documentação: Arquitetura do Servidor

(servidor_tcp.py)

Processamento de Regras, Estado do Jogo e Sincronização Multiprocessada

1. Visão Geral do Núcleo Backend

O arquivo **servidor_tcp.py** gerencia o estado centralizado e a aplicação estrita das regras do jogo de Truco Paulista. Ele funciona como o "State Master" (Árbitro) da partida. Nenhuma lógica de jogo ou validação de vitória roda de forma isolada nos clientes; tudo é processado sequencialmente pelo servidor.

A arquitetura divide-se em duas camadas fundamentais de envio de dados para os clientes:

- **Comunicação Direta Ponto a Ponto (Sockets TCP):** Canal bidirecional e privado. O servidor usa esta via para escutar os comandos assíncronos dos jogadores e para enviar informações estritamente privadas (como a distribuição inicial da mão de cartas de cada um).
- **Canal Broadcast de Baixa Latência (Shared Memory):** Um bloco de memória compartilhada nomeado ("MesaTruco") onde o servidor grava o estado atualizado da mesa. Os clientes locais leem esse bloco continuamente em intervalos fixos de tempo, reduzindo o tráfego na pilha de rede para atualizações puramente visuais.

2. Ciclo de Vida e Fluxo de Execução do Servidor

O fluxo de execução do servidor segue uma linha lógica estruturada desde a sua inicialização até o encerramento da partida:

1. **Alocação de Recursos Globais:** O servidor aloca um bloco de SharedMemory com tamanho suficiente (ex: 1024 bytes) e inicializa a estrutura principal através da classe Partida.
2. **Fase de Handshake e Entrada de Jogadores:** O servidor abre uma porta TCP (Socket de escuta) e entra em um loop aguardando conexões. Ele bloqueia o início do jogo até que o número exato de jogadores necessários (ex: 4 jogadores para duas duplas) se conecte com sucesso. Cada socket aceito é armazenado em uma lista indexada, associando a ordem de conexão ao ID do jogador no jogo.
3. **Distribuição Inicial de Cartas:** A rodada inicia limpando o estado anterior. O servidor usa as classes Baralho e Carta para embaralhar, definir o "Vira" e enviar de forma privada para cada socket a string correspondente às suas 3 cartas.
4. **Loop de Eventos do Jogo (Main Loop):** O servidor roda um loop contínuo processando o jogador da vez:
 - O estado atualizado é serializado em JSON e persistido na SharedMemory.

- O servidor monitora o socket do jogador cuja propriedade `partida.rodada_atual.vez` indica ser o atual a agir.
- Ao receber uma string de comando válida (Ex: "JOGAR: 1", "TRUCO"), o servidor aplica a mutação de estado.

5. **Avaliação de Quedas e Pontuação:** Após todos os jogadores válidos depositarem uma carta na mesa, a classe `Mesa` avalia quem jogou a carta de maior força (considerando as manilhas baseadas no Vira) e computa o vencedor daquela queda. O processo se repete até uma dupla atingir os pontos da rodada, atualizando o objeto `Placar` global.

3. Funções Críticas e Mecanismos do Sistema

Função	Lógica e Impacto no Fluxo do Jogo
<code>atualiza_memoria_compartilhada(shared_memory, partida)</code>	Monta um dicionário Python contendo dados públicos: o Placar atualizado, o ID do jogador da vez, o valor acumulado da aposta da rodada (1, 3, 6, 9, 12), a carta "Vira", e o mapa de cartas abertas ou encobertas presentes na mesa. Transforma essa estrutura em uma string JSON, converte para bytes e escreve diretamente no buffer da memória compartilhada. É chamada após cada alteração relevante de estado.
<code>verifica_e_envia_novas_cartas(conexoes, partida)</code>	Verifica se a rodada atual foi finalizada (se alguma dupla venceu duas quedas ou se houve situações de empate mapeadas). Se sim, o servidor inicia uma nova rodada interna na classe <code>Partida</code> , redefine o baralho, sorteia o novo Vira e despacha de forma privada por TCP a nova mão de cartas para os respectivos jogadores.
Loop de Tratamento de Comandos (Mensagens TCP)	O servidor realiza uma leitura não-bloqueante ou estruturada dos bytes trafegados no socket ativo. As strings são interpretadas com base em prefixos: "JOGAR: [index]": Remove a carta correspondente da mão do jogador e adiciona na mesa. "TRUCO": Altera o estado da rodada para <code>EstadoRodada.AGUARDANDO_RESPOSTA</code> e

congela o fluxo normal até a resposta do time adversário.

Gerenciamento de Escopo e Regras (Model-Driven Design)

É importante destacar que o arquivo `servidor_tcp.py` funciona apenas como o controlador de E/S (Entrada/Saída). A inteligência de jogo está distribuída nas classes de suporte: **Carta** processa a força relativa através do método `compara_cartas`; **Mesa** encapsula o dicionário de jogadas da queda; e **Rodada** rastreia a máquina de estados interna do Truco (se a partida está em modo normal ou aguardando aceite de aumento de pontos).

4. Mapeamento de Dependências de Negócio

O servidor interage diretamente com o ecossistema de objetos anexados:

- **Partida.py:** Controla a lista ordenada de objetos do tipo `Jogador`, gerencia a virada de rodadas e incrementa os pontos da dupla vencedora até que o limite de 12 pontos seja atingido, disparando a mensagem final "FIM_DE_JOGO" para todos os clientes.
- **Rodada.py:** Controla o valor de aposta corrente da rodada. Regula as nuances complexas de pontuação do Truco, como a "Mão de Onze" (onde a rodada já inicia valendo 3 pontos e possui regras de visualização de cartas restritas) e os critérios de desempate quando ocorrem quedas amarradas.
- **Carta.py:** Contém a lista estática com a ordem de força padrão do Truco Paulista ([4, 5, 6, 7, 8, 9, 10, 1, 2, 3]). É quem calcula dinamicamente qual valor representa a "Manilha" da rodada com base na operação matemática $(\text{indice_vira} + 1) \% \text{len}(\text{ordem_forca})$.